

JavaScript 기초 정리: 연산자, 조건문, 반복문, 함수, 스코프, 배열

JavaScript 문법은 하나씩 따로 배우면 단순해 보이지만, 실제 코드는 여러 개념이 함께 사용된다.

이번 글에서는 다음 흐름으로 JavaScript의 핵심 문법을 정리한다.

값을 계산한다	→ 연산자
상황을 판단한다	→ 조건문
반복해서 처리한다	→ 반복문
코드를 묶는다	→ 함수
변수 범위를 나눈다	→ 스코프
여러 값을 다룬다	→ 배열

1. 연산자

연산자는 값을 계산하거나 비교하고, 조건을 판단할 때 사용하는 기호다.

1.1 연산자 종류

종류	연산자	설명
산술 연산자	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>**</code>	사칙연산, 나머지, 거듭제곱
대입 연산자	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code>	변수에 값을 저장하거나 누적
비교 연산자	<code>==</code> , <code>===</code> , <code>!=</code> , <code>!==</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>	두 값을 비교
논리 연산자	<code>&&</code> , <code> </code> , <code>!</code>	AND, OR, NOT
증감 연산자	<code>++</code> , <code>--</code>	값을 1 증가 또는 감소
문자열 연산자	<code>+</code>	문자열 연결
삼항 연산자	<code>조건 ? 값1 : 값2</code>	조건에 따라 값 선택
타입 연산자	<code>typeof</code> , <code>instanceof</code>	자료형 확인
기타 연산자	<code>,</code> , <code>?.</code> , <code>??</code>	쉼표, 옵셔널 체이닝, null 병합

2. 비교 연산자

JavaScript에서 `==` 와 `===` 는 다르게 동작한다.

```
console.log(5 == "5")
console.log(5 === "5")
```

출력 결과

```
true
false
```

`==`는 값만 비교한다. 따라서 숫자 `5`와 문자열 `"5"`를 같다고 판단한다.

`===`는 값과 타입을 모두 비교한다. 숫자 `5`와 문자열 `"5"`는 타입이 다르기 때문에 `false`가 된다.

실제 개발에서는 예측 가능한 비교를 위해 `===`와 `!==`를 사용하는 것이 좋다.

```
console.log(10 !== "10")
console.log(10 !== "10")
```

```
false
true
```

3. 증감 연산자

증감 연산자는 값을 1 증가하거나 1 감소시킨다.

```
let count = 1

count++
console.log(count)

count--
console.log(count)
```

출력 결과

```
2
1
```

전위 증가와 후위 증가

증감 연산자는 변수 앞에 붙는지, 뒤에 붙는지에 따라 결과가 달라질 수 있다.

```
let count = 1

let result = ++count
console.log(result)
console.log(count)

result = count++
console.log(result)
console.log(count)
```

출력 결과

```
2
2
2
3
```

`++count`는 값을 먼저 증가시킨 뒤 사용한다.

`count++`는 현재 값을 먼저 사용한 뒤 값을 증가시킨다.

```
++count → 증가 후 사용
count++ → 사용 후 증가
```

4. 삼항 연산자

삼항 연산자는 조건에 따라 다른 값을 선택할 때 사용한다.

```
조건 ? 조건이_true일_때_값 : 조건이_false일_때_값
```

예제

```
let score = 85
let result = score >= 60 ? "합격" : "불합격"

console.log(result)
```

출력 결과

```
합격
```

간단한 조건 판단은 삼항 연산자로 짧게 작성할 수 있다. 하지만 조건이 복잡해지면 `if` 문을 사용하는 편이 더 읽기 좋다.

5. 타입 관련 연산자

5.1 `typeof`

`typeof` 는 값의 자료형을 문자열로 반환한다.

```
let num = 10

console.log(typeof num)
```

```
number
```

5.2 `instanceof`

`instanceof` 는 특정 객체가 어떤 생성자 또는 클래스의 인스턴스인지 확인한다.

```
let arr = []

console.log(arr instanceof Array)
```

```
true
```

배열은 객체이기 때문에 `typeof arr` 를 사용하면 `"object"` 가 나온다. 배열인지 확인하고 싶을 때는 `instanceof Array` 또는 `Array.isArray()` 를 사용할 수 있다.

```
console.log(typeof arr)
console.log(Array.isArray(arr))
```

```
object
true
```

6. 기타 연산자

6.1 쉼표 연산자

쉼표 연산자는 여러 표현식을 왼쪽부터 평가하고 마지막 값을 반환한다.

```
let x = (1, 2, 3, 10)

console.log(x)
```

```
10
```

일반적인 코드에서는 자주 사용하지 않지만, 이런 동작을 가진다는 점은 알아두면 좋다.

6.2 옵셔널 체이닝

옵셔널 체이닝 `?.`은 값이 `null` 또는 `undefined` 일 때 오류를 발생시키지 않고 `undefined`를 반환한다.

```
let user = null

console.log(user?.name)
```

```
undefined
```

만약 `user.name` 처럼 직접 접근하면 오류가 발생한다.

```
// console.log(user.name) // TypeError
```

6.3 null 병합 연산자

null 병합 연산자 `??`는 왼쪽 값이 `null` 또는 `undefined`이면 오른쪽 값을 반환한다.

```
let value = null
let resultValue = value ?? "기본값"

console.log(resultValue)
```

```
기본값
```

`0`, `false`, `""`는 값이 있는 것으로 취급한다.

```
console.log(0 ?? "기본값")
console.log(false ?? "기본값")
console.log("" ?? "기본값")
```

```
0
false
```

7. 조건문

조건문은 조건에 따라 다른 코드를 실행할 때 사용한다.

7.1 if

```
if (조건) {
    조건이 참일 때 실행할 코드
}
```

7.2 if / else

```
if (조건) {
    조건이 참일 때 실행할 코드
} else {
    조건이 거짓일 때 실행할 코드
}
```

7.3 if / else if / else

여러 조건을 순서대로 검사할 때 사용한다.

```
let age = 20

if (age > 19) {
    console.log("성인입니다")
} else if (age > 14) {
    console.log("청소년입니다")
} else if (age > 7) {
    console.log("어린이입니다")
} else {
    console.log("유아입니다")
}
```

출력 결과

```
성인입니다
```

조건문은 위에서 아래로 검사한다. 먼저 참이 되는 조건을 만나면 해당 블록을 실행하고 나머지 조건은 검사하지 않는다.

8. switch 문

`switch` 문은 하나의 값이 여러 경우 중 어디에 해당하는지 비교할 때 사용한다.

```
switch (값) {
  case 값1:
    실행할 코드
    break
  case 값2:
    실행할 코드
    break
  default:
    모든 case와 일치하지 않을 때 실행할 코드
}
```

MBTI 예제

```
let mbti = "ENFP"

switch (mbti) {
  case "INFJ":
    console.log("조용하지만 깊은 통찰력")
    break
  case "ENFP":
    console.log("열정 가득")
    break
  case "ISTJ":
    console.log("현실주의자")
    break
  case "ENTJ":
    console.log("리더십이 뛰어남")
    break
  default:
    console.log("등록되지 않은 MBTI")
}
```

출력 결과

열정 가득

`break` 는 해당 `case` 실행 후 `switch` 문을 빠져나가게 한다.

여러 case를 묶는 예제

```
const month = 6

switch (month) {
  case 1:
  case 3:
  case 5:
  case 7:
  case 8:
  case 10:
  case 12:
    console.log(`${month}월의 마지막 일자는 31일입니다.`)
    break
  case 2:
```

```

        console.log(`${month}월의 마지막 일자는 29일 또는 28일입니다.`)
        break
    case 4:
    case 6:
    case 9:
    case 11:
        console.log(`${month}월의 마지막 일자는 30일입니다.`)
        break
    default:
        console.log("1부터 12 사이의 월을 입력하세요.")
}

```

출력 결과

6월의 마지막 일자는 30일입니다.

여러 `case` 가 같은 결과를 가져야 한다면 위처럼 묶어서 작성할 수 있다.

9. 반복문

반복문은 같은 작업을 여러 번 실행할 때 사용한다.

9.1 `while` 문

조건식이 `true` 인 동안 코드를 반복한다.

```

while (조건식) {
    반복 실행할 코드
}

```

구구단 예제

```

let dan = 7
let i = 1

while (i <= 9) {
    console.log(`${dan} * ${i} = ${dan * i}`)
    i++
}

```

출력 결과

```

7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
...
7 * 9 = 63

```

`while` 문은 반복 횟수가 명확하지 않고 조건 중심으로 반복할 때 사용하기 좋다.

9.2 `do while` 문

`do while` 문은 조건과 상관없이 코드를 최소 한 번 실행한다.

```
let num = 10

do {
  console.log(`현재 num: ${num}`)
  num++
} while (num <= 9)
```

출력 결과

현재 num: 10

조건식 `num <= 9` 는 처음부터 `false` 다. 하지만 `do` 블록이 먼저 실행되므로 한 번은 출력된다.

9.3 for 문

반복 횟수가 정해져 있을 때 자주 사용한다.

```
for (초기식; 조건식; 증감식) {
  반복 실행할 코드
}
```

예제

```
for (let i = 1; i <= 5; i++) {
  console.log(`현재 i의 값: ${i}`)
}
```

출력 결과

현재 i의 값: 1
현재 i의 값: 2
현재 i의 값: 3
현재 i의 값: 4
현재 i의 값: 5

10. 중첩 반복문

반복문 안에 반복문을 작성할 수 있다.

```
for (let dan = 2; dan <= 9; dan++) {
  console.log(`${dan}단 구구단`)

  for (let i = 1; i <= 9; i++) {
    console.log(`${dan} * ${i} = ${dan * i}`)
  }
}
```

출력 결과 일부


```
2단 구구단
2 * 1 = 2
2 * 2 = 4
...
9단 구구단
9 * 9 = 81
```

바깥쪽 반복문은 단을 바꾸고, 안쪽 반복문은 각 단에서 1부터 9까지 곱한다.

11. break와 continue

11.1 break

break는 반복문을 즉시 종료한다.

```
for (let i = 1; i <= 10; i++) {
  if (i > 5) break
  console.log(`i의 값: ${i}`)
}
```

출력 결과

```
i의 값: 1
i의 값: 2
i의 값: 3
i의 값: 4
i의 값: 5
```

11.2 continue

continue는 현재 반복을 건너뛰고 다음 반복으로 넘어간다.

```
for (let i = 1; i <= 10; i++) {
  if (i % 3 === 0) {
    continue
  }

  console.log(`i의 값: ${i}`)
}
```

출력 결과

```
i의 값: 1
i의 값: 2
i의 값: 4
i의 값: 5
i의 값: 7
i의 값: 8
i의 값: 10
```

위 예제에서는 3의 배수를 출력하지 않는다.

12. 함수

함수는 특정 작업을 수행하는 코드를 하나의 이름으로 묶은 것이다.

```
function 함수명(매개변수1, 매개변수2) {  
  실행할 코드  
  return 반환값  
}
```

`return`을 생략하면 함수는 자동으로 `undefined`를 반환한다.

13. 함수 선언문

함수 선언문은 가장 기본적인 함수 작성 방식이다.

```
const result = add(3, 5)  
console.log(result)  
  
function add(a, b) {  
  return a + b  
}
```

출력 결과

8

함수 선언문은 호이스팅이 적용되므로 선언보다 먼저 호출할 수 있다.

호이스팅

호이스팅은 JavaScript가 코드를 실행하기 전에 선언을 해당 스코프의 위쪽으로 끌어올린 것처럼 처리하는 동작이다.

함수 선언문은 함수 전체가 끌어올려진 것처럼 동작한다. 반면 `var` 변수는 선언만 끌어올려지고 값은 할당되지 않아 `undefined` 상태가 될 수 있다.

14. 함수 표현식

함수 표현식은 함수를 변수에 저장하는 방식이다.

```
const add2 = function (a, b) {  
  return a + b  
}  
  
const result2 = add2(3, 5)  
console.log(result2)
```

출력 결과

8

함수 표현식은 변수에 함수가 할당된 뒤부터 사용할 수 있다.

15. 화살표 함수

화살표 함수는 함수를 더 짧게 작성하는 방식이다.

```
const add3 = (a, b) => a + b

console.log(add3(3, 5))
```

출력 결과

8

본문이 한 줄이고 바로 값을 반환한다면 {} 와 return 을 생략할 수 있다.

```
const square = number => number * number

console.log(square(5))
```

25

16. 즉시 실행 함수

즉시 실행 함수는 정의하자마자 바로 실행되는 함수다.

```
;(function () {
  console.log("즉시 실행됨")
})();
```

출력 결과

즉시 실행됨

앞 코드와 이어져 해석되는 문제를 줄이기 위해 앞에 세미콜론을 붙여 작성하는 경우가 많다.

17. 콜백 함수

콜백 함수는 다른 함수의 인자로 전달되어 나중에 실행되는 함수다.

```
function greet(callback) {
  callback("김사과")
}

greet(function (name) {
  console.log(`${name}님 안녕하세요`)
})

greet((name) => {
  console.log(`${name}님 안녕하세요`)
})
```

출력 결과

```
김사과님 안녕하세요
김사과님 안녕하세요
```

콜백 함수는 공통 흐름은 유지하면서 세부 동작만 바꾸고 싶을 때 유용하다.

18. 가변 매개변수

가변 매개변수는 함수에 전달되는 인자의 개수가 정해져 있지 않을 때 사용한다.

`...`을 사용하면 여러 값을 배열처럼 받을 수 있다.

```
function sum(...numbers) {
  let total = 0

  for (let num of numbers) {
    total += num
  }

  return total
}

console.log(sum(1, 2, 3))
console.log(sum(1, 20, 40, 26, 100))
```

출력 결과

```
6
187
```

19. 중첩 함수

중첩 함수는 함수 안에 정의된 함수다.

```
function printGreeting(name) {
  function message() {
    return `안녕하세요 ${name}님`
  }

  console.log(message())
}

printGreeting("김사과")
```

출력 결과

```
안녕하세요 김사과님
```

내부 함수는 외부 함수의 변수에 접근할 수 있다.

20. 고차 함수

고차 함수는 함수를 인자로 받거나 함수를 반환하는 함수다.

```
function createMultiplier(multiplier) {  
  return function (number) {  
    return number * multiplier  
  }  
}  
  
const multiplyBy5 = createMultiplier(5)  
  
console.log(multiplyBy5(5))
```

출력 결과

25

`createMultiplier(5)` 는 숫자에 5를 곱하는 함수를 만들어 반환한다.

21. 스코프

스코프(Scope)는 변수가 유효한 범위를 의미한다.

JavaScript는 변수를 찾을 때 현재 스코프에서 먼저 찾고, 없으면 바깥 스코프로 올라가며 검색한다. 이 구조를 스코프 체인이라고 한다.

21.1 전역 스코프

전역 스코프는 코드 어디에서든 접근 가능한 최상위 범위다.

```
var a = 1  
let b = 2  
const c = 3  
  
function func1() {  
  console.log(a, b, c)  
}  
  
func1()  
console.log(window.a)  
console.log(window.b)
```

출력 결과

1 2 3
1
undefined

브라우저 환경에서 전역에서 `var` 로 선언한 변수는 `window` 객체의 프로퍼티가 될 수 있다. 하지만 `let` 과 `const` 는 전역에 선언해도 `window` 의 프로퍼티로 붙지 않는다.

21.2 함수 스코프

`var`는 블록이 아니라 함수 단위 스코프를 가진다.

```
function func2() {
  if (true) {
    var x = 10
    let y = 20
  }

  console.log(x)
  // console.log(y) // ReferenceError
}

func2()
```

출력 결과

10

`x`는 `var`로 선언되었기 때문에 `if` 블록 밖에서도 접근할 수 있다. `y`는 `let`으로 선언되었기 때문에 블록 밖에서 접근할 수 없다.

21.3 블록 스코프

`let`과 `const`는 `{ }` 블록 단위로 유효 범위가 결정된다.

```
if (true) {
  let y = 20
  const z = 30

  console.log(y)
  console.log(z)
}

// console.log(y) // ReferenceError
// console.log(z) // ReferenceError
```

21.4 렉시컬 스코프

렉시컬 스코프는 함수가 어디에서 호출되었는지가 아니라 어디에 선언되었는지에 따라 상위 스코프가 결정되는 방식이다.

```
function func3() {
  const x = "outer"

  function inner() {
    console.log(x)
  }

  inner()
}

func3()
```

출력 결과

outer

`inner()` 는 `func3()` 안에서 선언되었기 때문에 `func3()`의 변수 `x`에 접근할 수 있다.

22. 배열

배열은 여러 개의 값을 하나의 변수에 순서대로 저장하는 자료구조다.

```
let 변수명 = [값1, 값2, 값3]
```

JavaScript 배열의 특징은 다음과 같다.

- 인덱스는 0 부터 시작한다.
- 여러 자료형을 함께 저장할 수 있다.
- 길이는 자동으로 관리된다.
- 배열도 객체다.
- 인덱스를 기준으로 값을 저장하고 접근한다.

22.1 배열 생성과 인덱싱

```
const user = [1, "apple", "김사과", 20, "서울 서초구"]

console.log(user)
console.log(user[0])
console.log(user[1])
console.log(user[2])
console.log(user[3])
console.log(user[4])
```

출력 결과

```
[1, "apple", "김사과", 20, "서울 서초구"]
1
apple
김사과
20
서울 서초구
```

22.2 배열 값 수정

```
user[4] = "서울 강남구"

console.log(user[4])
console.log(user.length)
```

출력 결과

```
서울 강남구
5
```

22.3 비어 있는 인덱스

JavaScript 배열은 중간 인덱스를 비워둔 상태로 값을 넣을 수 있다.

```
user[6] = "여자"

console.log(user)
console.log(user.length)
console.log(user[5])
```

출력 결과

```
[1, "apple", "김사과", 20, "서울 강남구", empty, "여자"]
7
undefined
```

5번 인덱스에는 값을 넣지 않았으므로 `undefined` 처럼 접근된다.

23. 배열 반복

배열의 모든 값을 처리할 때 반복문을 사용할 수 있다.

```
for (let i = 0; i < user.length; i++) {
  console.log(user[i])
}
```

배열 요소 자체가 필요하다면 `for...of` 를 사용할 수도 있다.

```
for (let value of user) {
  console.log(value)
}
```

24. 배열 메서드

배열은 값을 추가, 삭제, 변환하기 위한 다양한 메서드를 제공한다.

메서드	설명	원본 변경
<code>push()</code>	배열 끝에 요소 추가	O
<code>pop()</code>	배열 마지막 요소 제거	O
<code>unshift()</code>	배열 앞에 요소 추가	O
<code>shift()</code>	배열 첫 요소 제거	O
<code>forEach()</code>	각 요소를 순서대로 처리	X
<code>map()</code>	각 요소를 가공해 새 배열 반환	X
<code>filter()</code>	조건을 만족하는 요소만 새 배열로 반환	X
<code>reduce()</code>	배열을 하나의 값으로 누적 계산	X
<code>find()</code>	조건을 만족하는 첫 번째 요소 반환	X
<code>findIndex()</code>	조건을 만족하는 첫 번째 요소의 인덱스 반환	X
<code>includes()</code>	특정 값 포함 여부 반환	X
<code>indexOf()</code>	특정 값의 첫 인덱스 반환	X
<code>sort()</code>	배열 정렬	O
<code>reverse()</code>	배열 순서 반전	O
<code>slice()</code>	배열 일부를 잘라 새 배열 반환	X
<code>splice()</code>	요소 추가, 삭제, 교체	O
<code>join()</code>	배열을 문자열로 변환	X
<code>concat()</code>	배열을 합쳐 새 배열 반환	X
<code>flat()</code>	중첩 배열을 평탄화	X

25. 배열 메서드 예제

25.1 `push()` 와 `pop()`

```

user.push("ISTJ")
console.log(user)

let temp = user.pop()
console.log(user)
console.log(temp)

```

`push()` 는 배열 끝에 값을 추가한다. `pop()` 은 배열 마지막 값을 제거하고 제거한 값을 반환한다.

25.2 `shift()`

```
temp = user.shift()
console.log(user)
console.log(temp)
```

`shift()` 는 배열의 첫 번째 요소를 제거하고 제거한 값을 반환한다.

25.3 `concat()`

```
const profile = ["A형", "ISTJ"]
let result = user.concat(profile)

console.log(result)
```

`concat()` 은 두 배열을 합쳐 새 배열을 반환한다. 원본 배열은 변경하지 않는다.

25.4 `join()`

```
result = user.join("😎")

console.log(result)
console.log(typeof result)
```

`join()` 은 배열 요소를 하나의 문자열로 연결한다.

25.5 `sort()` 와 `reverse()`

```
const arr = ["apple", "banana", "orange", "Apple", "melon"]

arr.sort()
console.log(arr)

arr.reverse()
console.log(arr)
```

출력 결과

```
["Apple", "apple", "banana", "melon", "orange"]
["orange", "melon", "banana", "apple", "Apple"]
```

`sort()` 는 기본적으로 문자열 기준으로 정렬한다. 대문자와 소문자의 정렬 순서도 영향을 받는다.

숫자를 오름차순으로 정렬하려면 비교 함수를 전달한다.

```
const numbers = [10, 2, 30, 1]

numbers.sort((a, b) => a - b)
console.log(numbers)
```

```
[1, 2, 10, 30]
```

26. 예제를 연결해서 보기: 점수 관리 프로그램

지금까지 배운 내용을 하나의 짧은 예제로 연결해본다.

목표

학생들의 점수를 배열에 저장하고, 평균 점수와 합격 여부를 출력한다.

여기에는 다음 개념이 함께 사용된다.

- 배열
- 반복문
- 연산자
- 조건문
- 함수
- 스코프

예제 코드

```
const students = [
  { name: "김사과", score: 85 },
  { name: "반하나", score: 58 },
  { name: "오렌지", score: 92 },
]

function getAverageScore(studentList) {
  let total = 0

  for (let student of studentList) {
    total += student.score
  }

  return total / studentList.length
}

function printPassResult(studentList) {
  for (let student of studentList) {
    const result = student.score >= 60 ? "합격" : "불합격"
    console.log(`${student.name}: ${student.score}점, ${result}`)
  }
}

const average = getAverageScore(students)

console.log(`평균 점수: ${average}`)
printPassResult(students)
```

출력 결과

```
평균 점수: 78.33333333333333
김사과: 85점, 합격
반하나: 58점, 불합격
오렌지: 92점, 합격
```

코드 흐름

```
students 배열에 학생 정보 저장
↓
getAverageScore() 함수로 평균 계산
↓
printPassResult() 함수로 학생별 합격 여부 출력
↓
삼항 연산자로 합격 / 불합격 판단
```

`studentList`, `total`, `student`, `result` 같은 변수는 함수 또는 반복문 블록 안에서 선언되므로 필요한 범위 안에서만 사용된다. 이렇게 스코프를 잘 나누면 불필요한 전역 변수 사용을 줄일 수 있다.

27. 핵심 정리

연산자

- `===` 는 값과 타입을 모두 비교한다.
- `++count` 는 증가 후 사용하고, `count++` 는 사용 후 증가한다.
- `?.` 는 `null` 또는 `undefined` 접근 오류를 줄여준다.
- `??` 는 값이 `null` 또는 `undefined` 일 때 기본값을 사용할 수 있게 한다.

조건문

- 조건이 단순하면 삼항 연산자를 사용할 수 있다.
- 여러 범위를 판단할 때는 `if / else if / else` 를 사용한다.
- 하나의 값이 여러 경우 중 어디에 해당하는지 판단할 때는 `switch` 문이 유용하다.

반복문

- 조건 중심 반복은 `while` 을 사용할 수 있다.
- 최소 한 번 실행해야 하면 `do while` 을 사용할 수 있다.
- 반복 횟수가 명확하면 `for` 문이 적합하다.
- `break` 는 반복문 종료, `continue` 는 현재 반복 건너뛰기다.

함수

- 함수는 반복되는 코드를 묶고 재사용하기 위해 사용한다.
- 함수 선언문은 호이스팅이 적용된다.
- 함수 표현식과 화살표 함수는 변수에 함수를 저장하는 방식이다.
- 콜백 함수는 다른 함수에 전달되어 나중에 실행된다.
- 고차 함수는 함수를 인자로 받거나 함수를 반환한다.

스코프

- 스코프는 변수가 유효한 범위다.
- `var` 는 함수 스코프를 가진다.
- `let` 과 `const` 는 블록 스코프를 가진다.
- 내부 함수는 선언된 위치를 기준으로 바깥 변수에 접근한다.

배열

- 배열은 여러 값을 순서대로 저장한다.
- 인덱스는 0 부터 시작한다.
- `push()`, `pop()`, `shift()` 등으로 값을 추가하거나 제거할 수 있다.
- `concat()`, `join()`, `sort()`, `reverse()` 등으로 배열을 조작할 수 있다.
- 숫자 정렬은 비교 함수를 사용하는 것이 안전하다.